TEMA 4

Las técnicas no supervisadas más comunes son:

- Clústering. Útil para análisis de datos, segmentación, sistemas de recomendación, motores de búsquedas, reducción de dimensionalidad.
- Detección de anomalías o valores atípicos.
- Estimación de densidad. Consiste en estimar la función de densidad de probabilidad del proceso aleatorio que generó los datos.

1. Clustering

Consiste en la tarea de identificar instancias similares y asignarlas a un mismo clúster. El clústering es lo mismo que la clasificación solo que de forma no supervisada: no se tienen las etiquetas de las instancias.

Se suele utilizar para:

- Segmentación de clientes/imágenes.
- Análisis de datos. Separa los datos según características.
- Reducción de dimensionalidad.
- Ingeniería de características. Para añadir nuevas características a los datos que ayude en otros problemas.
- Detección de anomalías.
- Aprendizaje semi-supervisado. Crea pseudoetiquetas para un conjunto de datos no etiquetado, de forma que pueda ser usado por un algoritmo de clasificación.
- Motores de búsqueda.

No existe una definición específica de clúster: desde las instancias a x distancia de un centroide a regiones densas del espacio.

1.1 K-medias

Este algoritmo consiste en agrupar las instancias buscando los K puntos que generen la mejor agrupación posible, es decir, haciendo mínima la distancia entre las instancias y el K punto correspondiente.

```
kmeans = KMeans(n_clusters=k, random_state=42)

y_pred = kmeans.fit_predict(X)
```

Si se quiere una puntuación de pertenencia a cada clúster en lugar del ID del clúster en sí:

```
kmeans.transform(X)
```

El algoritmo funciona de la siguiente forma: se establecen k puntos aleatorios (centroides) en el espacio y se etiquetan los puntos según la distancia a los diferentes centroides. A partir de este etiquetado se calcula el punto medio entren todas las instancias de la misma etiqueta. Con los nuevos centroides se vuelven a calcular las etiquetas de las instancias según la distancia. Se vuelve a actualizar los centroides y así sucesivamente.

La complejidad computacional suele ser lineal en función del número de instancias (m), el número de centroides (k) y el número de características (k).

El algoritmo garantiza la convergencia, pero no a la solución correcta. Se puede ejecutar el algoritmo varias veces para encontrar la mejor solución (n_init en scikit). Para ello se usa la inercia, que es la suma de las distancias cuadradas entre las instancias y el centroide más cercano. Esta inercia no se puede usar para decidir el mejor valor de K, sino para elegir el mejor modelo dentro dado un valor K. Para decidir el mejor valor de K se usa la silueta, que se define en base a la distancia media a las instancias de su clúster y la distancia media a las instancias del clúster más cercano. La silueta tiene un valor entre -1 (la instancia se ha asignado incorrectamente) y 1 (la instancia se ha asignado a su propio cluster y está bastante interno). A más cercano a 1, mejor.

Existe una versión mejorada del K-medias que consiste en realizar una inicialización de los centroides de manera óptima: a partir de un centroide elegido aleatoriamente, se eligen el resto a partir de la asignación de una probabilidad a cada instancia. Esta probabilidad se calcula en función de la distancia entre esa instancia y el centroide más cercano. De esta forma, el siguiente centroide estará lo más alejado posible del primer centroide. Este proceso se repite sucesivamente par los k-2 centroides faltantes. Este algoritmo es el que está implementado en la clase kmeans de scikit.

Otras versiones son:

- La propuesta por Elkan: en grandes datasets acelera el algoritmo evitando cálculos inncesarios, pero en algunos casos puede ralentizarlo significativamente.
- La propuesta de Sculley: para grandes datasets que no quepan en memoria se pueden sacar lotes más pequeños e ir calculando los centroides con esos lotes.

Este algoritmo no funciona muy bien cuando se tiene clusters de diferentes tamaños, densidades o formas no esféricas. Es importante estandarizar los datos para evitar esas formas en la medida que se pueda.

1.1.1 Usos

En lo relacionado con la segmentación de imágenes puede usarse para diferentes variantes:

- Para la segmentación de colores. Es decir, agrupar los píxeles por similitud de colores para analizar información que pueda haber.
- Para la segmentación semántica. Para buscar píxeles que sean parte del mismo objeto de la imagen.
- Para la segmentación de instancias.

1.2 DBSCAN

Este algoritmo define los clústeres como regiones continuas de alta densidad. Para cada instancia el algoritmo cuenta cuantas instancias están a una distancia e. Este conjunto de instancias determina el vecindario-e. Si este conjunto, junto con la propia instancia, tiene un mínimo de instancias, se dice que es una instancia núcleo, es decir, es una instancia que está dentro de la región densa. Todas las instancias de ese conjunto forman parte del mismo clúster. Este algoritmo se va haciendo para todas las instancias y se va determinando las regiones. En caso de que una instancia no sea una instancia núcleo y no pertenezca a ningún vecindario-e, se dice que esa instancia es una anomalía.

Este algoritmo funciona bien para clústeres separados por regiones de baja densidad.

Este algoritmo no tiene la función de predecir, por lo que se suele usar simplemente para determinar los clústeres y las anomalías, aunque se pueden usar las instancias núcleo para implementar un clasificador KNN.

La complejidad computacional de este algoritmo es $O(m^2n)$, por lo que para conjuntos muy grandes no es muy adecuado.

1.3 Mezclas gaussianas

Los modelos de mezclas gaussianas son modelos generativos, es decir, que pueden generar nuevas instancias a partir del ajuste que se haga. Estos modelos asumen que las instancias están generadas a partir de una mezcla de varias distribuciones gaussianas cuyos parámetros son desconocidos. La versión más simple asume K distribuciones gaussianas, que asume que los datos son generados tal que:

- Para cada instancia se toma un clúster aleatorio de los K. La probabilidad de seleccionar el clúster j es el peso del clúster ϕ_j. El índice del clúster elegido, z_i, se le asigna a la instancia i.

- Si la instancia i se asigna al clúster j ($z_i = j$), entonces la ubicación de la instancia se muestrea aleatoriamente siguiendo $N(\mu_j, \Sigma_j)$.

Se trata de un algoritmo muy parecido al K-medias: los centroides serían μ_j, mientras que Σ_j determinan la orientación, tamaño y forma de los clústeres, así como ϕ_j determina los pesos de cada distribución. La principal diferencia es que para actualizar los parámetros de las distribuciones se toman todas las instancias, cada una ponderada con la probabilidad de pertenecer a esa distribución.

La forma de las distribuciones se puede personalizar para que se ajuste mejor a los datos. Las disponibles son: esférica (spherical), diag (los clústeres tienen forma elipsoidal, pero de forma paralela al eje de coordenadas) o tie (forma elipsoidal independiente a los ejes). Por defecto, GaussianMixture tiene el hiperparámetro covariance_type a full, que permite que los clústeres sean de cualquier forma independientemente los unos de los otros.

En el caso de las distribuciones esféricas y elipsoidales paralelas al eje, la complejidad es de O(kmn) asumiendo que los datos tienen estructura de clúster. Para la elipsoidal o la "libre" sería $O(kmn^2+kn^3)$.

La detección de anomalías con este modelo consistirá en buscar las instancias que se encuentren en áreas con una probabilidad menor al tanto por ciento que se quiera. Debido a que el modelo usa todos los datos para el entrenamiento, las distribuciones se ajustarán intentando incluir a los valores anómalos.

La selección del número de clústeres a utilizar se realiza usando el criterio de información bayesiano (BIC) y el criterio de información de Akaike (AIC). El AIC tiene a seleccionar modelos más complejos, pero sin que los datos se ajusten bien (especialmente en datasets grandes), mientras que el BIC tiene a seleccionar modelos más simples. Si se computa los criterios para cada valor de K, es probable encontrar un mínimo que sea el valor óptimo de K.